

Constraint – Based Problem Solving

1. University Exam Timetable Scheduling A university needs to schedule 3 subjects (S1, S2, S3) into 3 time slots (Morning, Afternoon, Evening).

Constraints:

- S1 cannot be scheduled in the Evening.
- S2 must be before S3.
Only one subject per time slot.
- S3 cannot be scheduled in the Morning.
Every subject must be assigned exactly one slot.

Task: Apply the Backtracking Search Algorithm with all intermediate steps and constraint checking. Write the final timetable. give solution for the above question

SOLUTION:

Problem Statement

A university needs to schedule **3 subjects (S1, S2, S3)** into **3 time slots:**

- Morning (M)
- Afternoon (A)
- Evening (E)

Constraints

1. S1 cannot be scheduled in the **Evening**.
2. S2 must be scheduled **before** S3 (**Morning < Afternoon < Evening**).
3. Only **one subject** can be assigned to each time slot.
4. S3 cannot be scheduled in the **Morning**.
5. Every subject must be assigned **exactly one** time slot.

Step 1: Define Variables and Domains

Variable Possible Time Slots

S1	Morning, Afternoon
S2	Morning, Afternoon, Evening
S3	Afternoon, Evening

Step 2: Initial State

No assignments have been made.

Subject Assigned Time Slot

S1 —

S2 —

S3 —

Step 3: Backtracking Search Process

Iteration 1: Assign S1

Try Morning

Constraint Check:

- S1 cannot be Evening ✓
- Morning is available ✓

Assignment

Subject Time Slot

S1 Morning

S2 —

S3 —

No constraint is violated.

Iteration 2: Assign S2

Try Morning

Morning is already occupied by S1.

Constraint Violated

✗ Only one subject per time slot.

Reject this assignment.

Try Afternoon

Constraint Check:

- Afternoon is free ✓
- S2 can still be before S3 ✓

Assignment

Subject Time Slot

S1 Morning

S2 Afternoon

S3 —

No constraint is violated.

Iteration 3: Assign S3

Try Morning

Constraint Check:

✗ S3 cannot be scheduled in Morning.

Reject.

Try Afternoon

Constraint Check:

✗ Afternoon is already occupied by S2.

Reject.

Try Evening

Constraint Check:

- Evening is free ✓
- S3 is not Morning ✓
- S2 (Afternoon) is before S3 (Evening) ✓

Assignment

Subject Time Slot

S1 Morning

S2 Afternoon

S3 Evening

All constraints are satisfied.

Constraint Verification

Constraint

Verification

Status

S1 cannot be Evening

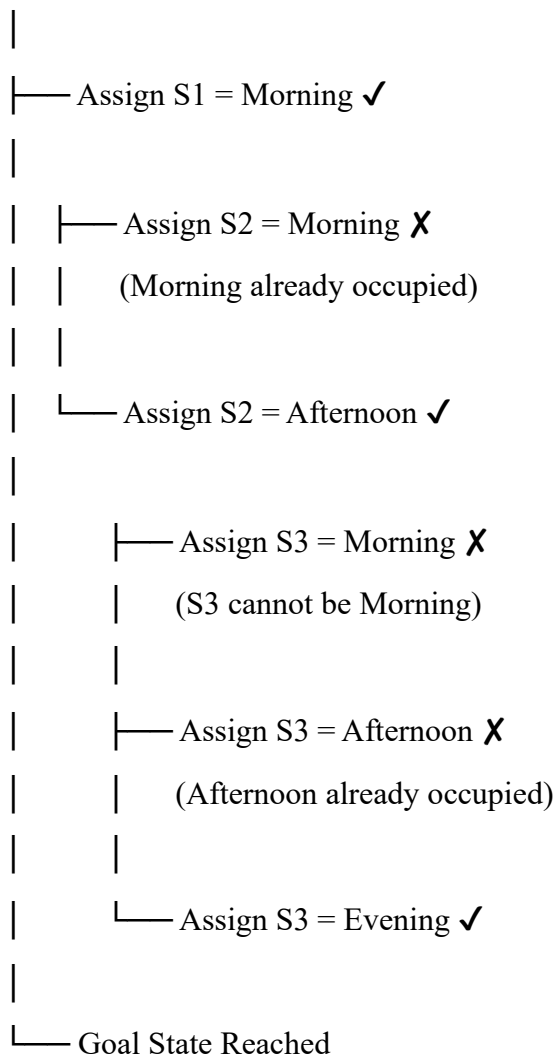
S1 = Morning

✓ Satisfied

Constraint	Verification	Status
S2 before S3	Afternoon before Evening	✓ Satisfied
One subject per slot	All slots contain only one subject	✓ Satisfied
S3 cannot be Morning	S3 = Evening	✓ Satisfied
Every subject assigned exactly one slot S1, S2, S3 each assigned once		✓ Satisfied

Backtracking Search Tree

Start



Final Timetable

Time Slot Subject

Morning S1

Afternoon S2

Evening S3

Algorithm Used (Backtracking Search)

1. Start with an empty timetable.
2. Assign a time slot to the first subject.
3. Check all constraints after each assignment.
4. If any constraint is violated, reject that assignment and try the next available time slot (**backtracking**).
5. Continue until all subjects are assigned valid time slots.
6. Stop when a complete valid timetable is obtained.

CONCLUSION:

Using the Backtracking Search Algorithm, each subject was assigned a time slot while checking all constraints at every step. Invalid assignments were rejected immediately, and valid assignments were retained. The final timetable obtained is:

Subject Assigned Time Slot

S1 Morning

S2 Afternoon

S3 Evening

Hence, the final valid exam timetable is:

- Morning → S1
- Afternoon → S2
- Evening → S3

This solution satisfies all the given constraints and represents the correct application of the Backtracking Search Algorithm.

2. Employee Shift Scheduling Assign 3 employees (E1, E2, E3) to Morning, Afternoon, Night shifts.

Constraints:

- E1 cannot work Night.
- E2 must work before E3.
One employee per shift.
- E3 cannot work Morning.
Every employee must have one shift.

Task: Use Backtracking Search and show every assignment and backtracking step. give clear and perfect answer for the above question

SOLUTION:

Given

Employees

- E1
- E2
- E3

Shifts

- Morning (M)
- Afternoon (A)
- Night (N)

Constraints

1. E1 cannot work Night.
2. E2 must work before E3 (Morning < Afternoon < Night).
3. Only one employee per shift.
4. E3 cannot work Morning.
5. Every employee must be assigned exactly one shift.

Backtracking Search Steps

Step 1: Assign E1

Try Morning ✓

Employee Shift

E1 Morning

E2 —

E3 —

Constraint satisfied.

Step 2: Assign E2

- Morning ✗ (Already occupied)
- Afternoon ✓

Employee Shift

E1 Morning

E2 Afternoon

E3 —

Constraint satisfied.

Step 3: Assign E3

- Morning ✗ (Not allowed)
- Afternoon ✗ (Already occupied)
- Night ✓

Employee Shift

E1 Morning

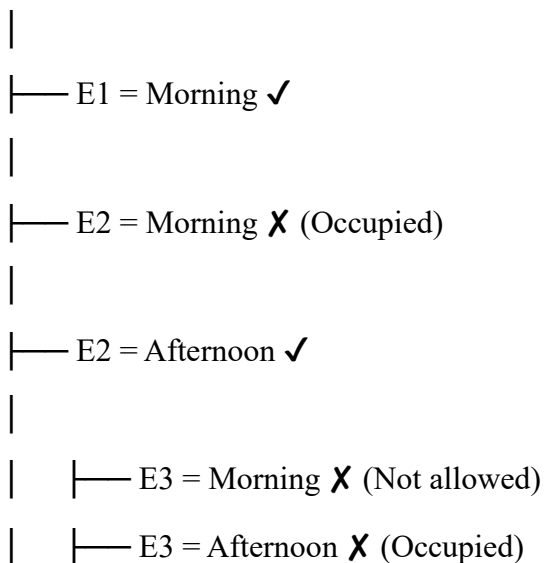
E2 Afternoon

E3 Night

All constraints are satisfied.

Backtracking Search Tree

Start



| └─ E3 = Night ✓

|

└─ Solution Found

Constraint Verification

Constraint	Status
E1 is not assigned Night	✓
E2 is before E3	✓ (Afternoon < Night)
One employee per shift	✓
E3 is not assigned Morning	✓
Every employee has one shift	✓

Final Shift Schedule

Shift	Employee
Morning	E1
Afternoon	E2
Night	E3

Conclusion:

Using the Backtracking Search Algorithm, the valid shift assignment is:

- Morning → E1
- Afternoon → E2
- Night → E3

This schedule satisfies all the given constraints.

3. Meeting Room Scheduling

Assign 3 meetings (M1, M2, M3) to 3 rooms (R1, R2, R3).

Constraints:

- M1 cannot use Room 3.

- M2 must be scheduled before M3.
- One meeting per room.
- M3 cannot use Room 1.
- Every meeting gets exactly one room.

Task:

Apply **Backtracking Search** and show all intermediate assignments, constraint checks, and the final schedule.

SOLUTION:

Given

Meetings

- M1
- M2
- M3

Rooms

- R1
- R2
- R3

Constraints

1. M1 cannot use Room R3.
2. M2 must be scheduled before M3.
3. Only one meeting per room.
4. M3 cannot use Room R1.
5. Every meeting must be assigned exactly one room.

Backtracking Search Steps

Step 1: Assign M1

Try Room R1 ✓

Meeting Room

M1	R1
M2	—
M3	—

Constraint satisfied.

Step 2: Assign M2

- Room R1 ✗ (Already occupied)
- Room R2 ✓

Meeting Room

M1 R1

M2 R2

M3 —

Constraint satisfied.

Step 3: Assign M3

- Room R1 ✗ (Not allowed)
- Room R2 ✗ (Already occupied)
- Room R3 ✓

Meeting Room

M1 R1

M2 R2

M3 R3

All constraints are satisfied.

Backtracking Search Tree

Start

|

|— M1 = R1 ✓

|

|— M2 = R1 ✗ (Occupied)

|

|— M2 = R2 ✓

|

| |— M3 = R1 ✗ (Not allowed)

```

|   |—— M3 = R2 ✗ (Occupied)
|   |—— M3 = R3 ✓
|
|—— Solution Found

```

Constraint Verification

Constraint	Status
M1 is not assigned Room R3	✓
M2 is scheduled before M3	✓
One meeting per room	✓
M3 is not assigned Room R1	✓
Every meeting has one room	✓

Final Meeting Schedule

Room Meeting

R1 M1

R2 M2

R3 M3

CONCLUSION:

Using the Backtracking Search Algorithm, the valid room assignment is:

- Room R1 → M1
- Room R2 → M2
- Room R3 → M3

All the given constraints are satisfied, so this is the final valid meeting room schedule.

4. Fire Rescue Robot

Robot moves inside a burning building.

Constraints:

- Move Up, Down, Left, Right.
- Fire zones cannot be crossed.
- Smoke areas cost 2.
- Safe cells cost 1.
- Use Uniform Cost Search.

Task:

Show priority queue updates, node expansion, optimal path, and total cost.

SOLUTION:

Given

Movement

- Up, Down, Left, Right

Constraints

- Fire zones (F) cannot be crossed.
- Safe cell (S) cost = 1
- Smoke cell (M) cost = 2

Building Layout

S S M

F S M

S S G

- Start = S (Top-left)
- Goal = G (Bottom-right)
- F = Fire (Blocked)

Step 1: Priority Queue

Priority Queue Cost

Start (0,0) 0

Step 2: Node Expansion

Expand Start (0,0)

Possible moves:

- Right → (0,1) Safe → Cost = 1 ✓

- Down → Fire ✗

Priority Queue

Node Cost

(0,1) 1

Expand (0,1)

Possible moves:

- Right → Smoke → Cost = 3 ✓
- Down → Safe → Cost = 2 ✓

Priority Queue

Node Cost

(1,1) 2

(0,2) 3

Expand (1,1)

Possible moves:

- Down → Safe → Cost = 3 ✓

Priority Queue

Node Cost

(0,2) 3

(2,1) 3

Expand (2,1)

Possible moves:

- Right → Goal → Cost = 4 ✓

Priority Queue

Node Cost

Goal (2,2) 4

Goal reached.

Optimal Path

Start

↓

(0,1)

↓

(1,1)

↓

(2,1)

→

Goal

Path Cost

Cell Cost

Start 0

Safe 1

Safe 1

Safe 1

Goal 1

Total Cost = 4

CONCLUSION:

- Algorithm Used: Uniform Cost Search (UCS)
- Optimal Path: Start → (0,1) → (1,1) → (2,1) → Goal
- Total Cost = 4
- UCS selects the path with the minimum cumulative cost while avoiding fire zones.

5. Classroom Allocation

Assign **3 classrooms (C1, C2, C3)** to **3 departments (D1, D2, D3)**.

Constraints:

- D1 cannot use C3.
- D2 must be assigned before D3.

- One department per classroom.
- D3 cannot use C1.
- Every department gets exactly one classroom.

Task:

Solve using **Backtracking Search** with intermediate steps.

SOLUTION:

Given

Classrooms

- C1
- C2
- C3

Departments

- D1
- D2
- D3

Constraints

1. D1 cannot use C3.
2. D2 must be assigned before D3.
3. Only one department per classroom.
4. D3 cannot use C1.
5. Every department must be assigned exactly one classroom.

Backtracking Search Steps

Step 1: Assign D1

Try C1 ✓

Department Classroom

D1 C1

D2 —

D3 —

Constraint satisfied.

Step 2: Assign D2

- C1 ✗ (Already occupied)
- C2 ✓

Department Classroom

D1 C1

D2 C2

D3 —

Constraint satisfied.

Step 3: Assign D3

- C1 ✗ (Not allowed)
- C2 ✗ (Already occupied)
- C3 ✓

Department Classroom

D1 C1

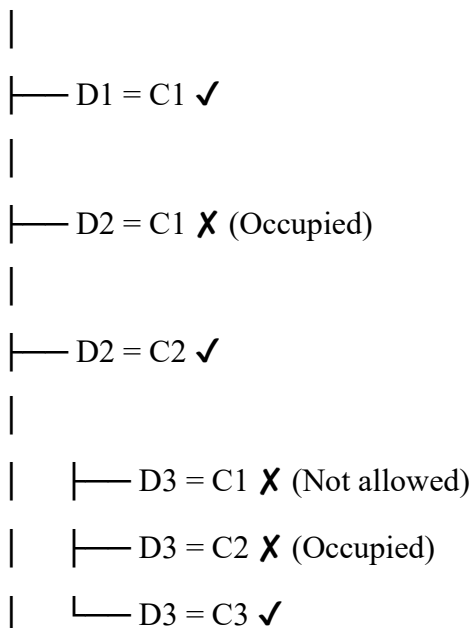
D2 C2

D3 C3

All constraints are satisfied.

Backtracking Search Tree

Start



|

└─ Solution Found

Constraint Verification

Constraint	Status
D1 is not assigned C3	✓
D2 is assigned before D3	✓
One department per classroom	✓
D3 is not assigned C1	✓
Every department has one classroom	✓

Final Classroom Allocation

Classroom Department

C1	D1
C2	D2
C3	D3

CONCLUSION:

Using the Backtracking Search Algorithm, the valid classroom allocation is:

- C1 → D1
- C2 → D2
- C3 → D3

All the given constraints are satisfied, making this the correct final allocation.